

AI Capstone HW4

112550169 潘仰祐

Task 1

your_fk() Explanation

In your_fk(), first I defined an helper function that generates the transformation in Classic D-H procedure. Those rotations and transformations can be represented as homogeneous coordinate matrices and can be multiply together into a single matrix.

```
def get_dh_matrix(q, d, a, alpha):  
    # Classic DH's transformation order:  
    # 1. Rotation  $q$  about Z axis  
    # 2. Translation  $d$  along Z axis  
    # 3. Translation  $a$  along X axis  
    # 4. Rotation  $\alpha$  about X axis  
  
    return np.array([  
        [np.cos(q), -np.sin(q)*np.cos(alpha), np.sin(q)*np.sin(alpha), a*np.cos(q)],  
        [np.sin(q), np.cos(q)*np.cos(alpha), -np.cos(q)*np.sin(alpha), a*np.sin(q)],  
        [0, np.sin(alpha), np.cos(alpha), d],  
        [0, 0, 0, 1]  
    ])
```

Then, I implemented the core of Classic D-H procedure. I found it more easy to think of it as "creating local coordinate systems (frames)" for each joints because [as the 3D animation mentioned](#), the origins may not be within the physical actuators.

```
A_list = [A]  
  
# For the 6 joints, calculate their local (frame) coord sys one by one  
for i in range(6):  
    # Get the info of i-th joint  
    q_i = q[i] # often called theta  
    d_i = DH_params[i]['d']  
    a_i = DH_params[i]['a']  
    alpha_i = DH_params[i]['alpha']  
  
    # make a dh matrix T_i  
    T_i = get_dh_matrix(q_i, d_i, a_i, alpha_i)  
  
    # calculate the next coord system:  
    # relative to current coord system -> relative to base coord system  
    A = A @ T_i  
  
    # store the coord system  
    A_list.append(A)
```

The part for Jacobian will be mentioned in Task 2.

Classic D-H Convention vs. Modified (Craig's) D-H Convention

I found it more easier for me to understand the differences through pseudo codes that transforms the physical positions of joints from the local coordinate system (bind to joints; their z-axes are the rotational centre axes of joints) to the base coordinate system.

```
# Modified D-H
A_list = [A_base]
global_pos = []
for i in range(6):
    T[i] = modified_dh(q[i], d[i], a[i], alpha[i])
    A = A_list[-1] @ T[i]

    A_list.append(A) # i.e. A_list[i+1] := A

    local_pos = ... # load it from URDF, etc.
    global_pos.append(A @ local_pos) # i.e.,
    # global_pos[i] := A_list[i+1] @ local_pos

def modified_dh(q, d, a, alpha):
    # Modified DH's transformation order:
    # 1. Translation $a$ along X axis
    # 2. Rotation $alpha$ about X axis
    # 3. Rotation $q$ about Z axis
    # 4. Translation $d$ along Z axis
    return Mat(1.) @ Mat(2.) @ Mat(3.) @ Mat(4.)
```

```
# Classic D-H
A_list = [A_base]
global_pos = []
for i in range(6):
    T[i] = classic_dh(q[i], d[i], a[i], alpha[i])
    A = A_list[-1] @ T[i]

    A_list.append(A) # A is A_list[i+1]

    local_pos = ... # load it from URDF, etc.
    global_pos.append(A_list[i] @ local_pos) # i.e.,
    # global_pos[i] := A_list[i] @ local_pos

def classic_dh(q, d, a, alpha):
    # Classic DH's transformation order:
    # 1. Rotation $q$ about Z axis
    # 2. Translation $d$ along Z axis
    # 3. Translation $a$ along X axis
    # 4. Rotation $alpha$ about X axis
    return Mat(1.) @ Mat(2.) @ Mat(3.) @ Mat(4.)
```

But they share the same spirits: aligning the x-axis to the common perpendicular segment between the two z-axes of neighbour revolute joints. And therefore, the D-H parameters are indeed very similar, with a slight offset and compensation by zeros. For example, extracting and modifying the parameters of ur5 from fk.py:

Classic D-H Parameter table

Index i	θ_i	d_i	a_i	α_i
1	q_1	0.0892	0	$\pi/2$
2	q_2	0	-0.425	0
3	q_3	0	-0.392	0
4	q_4	0.1093	0	$\pi/2$
5	q_5	0.09475	0	$-\pi/2$
6	q_6	0.2023	0	0

Modified D-H Parameter table

Index i	a_i	α_i	θ_i	d_i
1	0	0	q_1	0.0892
2	0	$\pi/2$	q_2	0
3	-0.425	0	q_3	0
4	-0.392	0	q_4	0.1093
5	0	$\pi/2$	q_5	0.09475
6	0	$-\pi/2$	q_6	0.2023

D-H Table for Franka (7 DoF)

Given the Classic D-H convention following this procedure at step i :

1. Move along Z-axis by d and rotate about Z-axis by θ , to make X-axis align to the common perpendicular segment. (In 3D space, around the same axis, the order between rotation and translations are interchangeable.)
2. Move along X-axis by a and rotate about Z-axis by α to align the Z-axes.

Assume the values $\{d_i: 1 \leq i \leq 7\}$, $\{\alpha_i: 1 \leq i \leq 7\}$, and $\{a_i: 1 \leq i \leq 7\}$ are all positive values and we need to add signs to indicating the translation and rotational directions. Then, the table for Franka's setting will be as the below (left).

Index i	d	α	a	θ	Index i	d	α	a	θ
1	d_1	$\pi/2$	$a_1=0$	θ_1	1	0.3330	$\pi/2$	0	θ_1
2	$d_2=0$	$-\pi/2$	$a_2=0$	θ_2	2	0	$-\pi/2$	0	θ_2
3	d_3	$\pi/2$	a_3	θ_3	3	0.3160	$\pi/2$	0.0825	θ_3
4	$d_4=0$	$-\pi/2$	$-a_4$	θ_4	4	0	$-\pi/2$	-0.0825	θ_4
5	d_5	$\approx \pi/2$	$a_5=0$	θ_5	5	0.3840	$\pi/2$	0	θ_5
6	$d_6=0$	$\approx \pi/2$	a_6	θ_6	6	0	$\pi/2$	0.088	θ_6
7	d_7	0	0	θ_7	7	0.107	0	0	θ_7

If we put the real values of Franka's D-H parameters online ([ref1](#), [ref2](#)), we can get another table as the above (right).

- Notice that in ref1, the tables is a Modified D-H table, and the flange and joint 7 are splitted.
- Notice that in both refs, the frames have different local coordinate systems and thus the signs are different in joint 1 and 2.

Task 2

your_ik() implementation

First, to form a Jacobian matrix indicating the instant velocity (linear and angular), we need to gather the information from the frames (local coordinate systems) during the FK process. Thus, I modified your_fk() to just calculate it there. I follows the formulae mentioned by Professor Yi-Ting's [Lecture 8](#), which calculates the linear velocity part of Jacobian with a cross product of z-axis vector (the rotational center) and a vector from current frame to end effector, both are represented in the base coordinate system.

```

## Calc Jacobian
# Jacobian is a 6x6 matrix indicating if we move each joints, how much
# do they contribute such that we can get closer to the target pose.
# jacobian[0:3][i]: The linear velocity (x, y, z) for joint_i
# jacobian[4:6][i]: The angular velocity that joint_i rotates

p_end = A_list[-1][:3, 3]
for i in range(6):
    # get the current pose of coord_i (effectively equivalent to joint_i)
    A_i = A_list[i]
    z_axis = A_i[:3, 2] # the vector parallel to rotational center axis
    p_i = A_i[:3, 3]    # the coord sys's origin's position rel to base

    # get the linear velocity
    J_v = np.cross(z_axis, (p_end - p_i))

    # get the angular velocity
    J_w = z_axis

    jacobian[:3, i] = J_v
    jacobian[3:, i] = J_w

```

Then, the list of thetas (list of q) is calculated by iteratively optimizing the error between the new_pose and current_pose of the end effector. This is the Jacobian Pseudo-Inverse method.

```

new_pose = np.asarray(new_pose)
dh_params = get_ur5_DH_params()

step_size = 0.1
for _ in range(max_iters):
    # get the current pose & jacobian matrix
    curr_pose, J = your_fk(dh_params, tmp_q, base_pos)

    # get the error
    pos_error = new_pose[:3] - curr_pose[:3]

    rotMat_target = R.from_quat(new_pose[3:]).as_matrix()
    rotMat_curr = R.from_quat(curr_pose[3:]).as_matrix()
    rotMat_error = rotMat_target @ rotMat_curr.T
    rot_error = R.from_matrix(rotMat_error).as_rotvec()

    error = np.concatenate([pos_error, rot_error])

    # check if the end pose is reached
    if np.linalg.norm(error) < stop_thresh:
        break

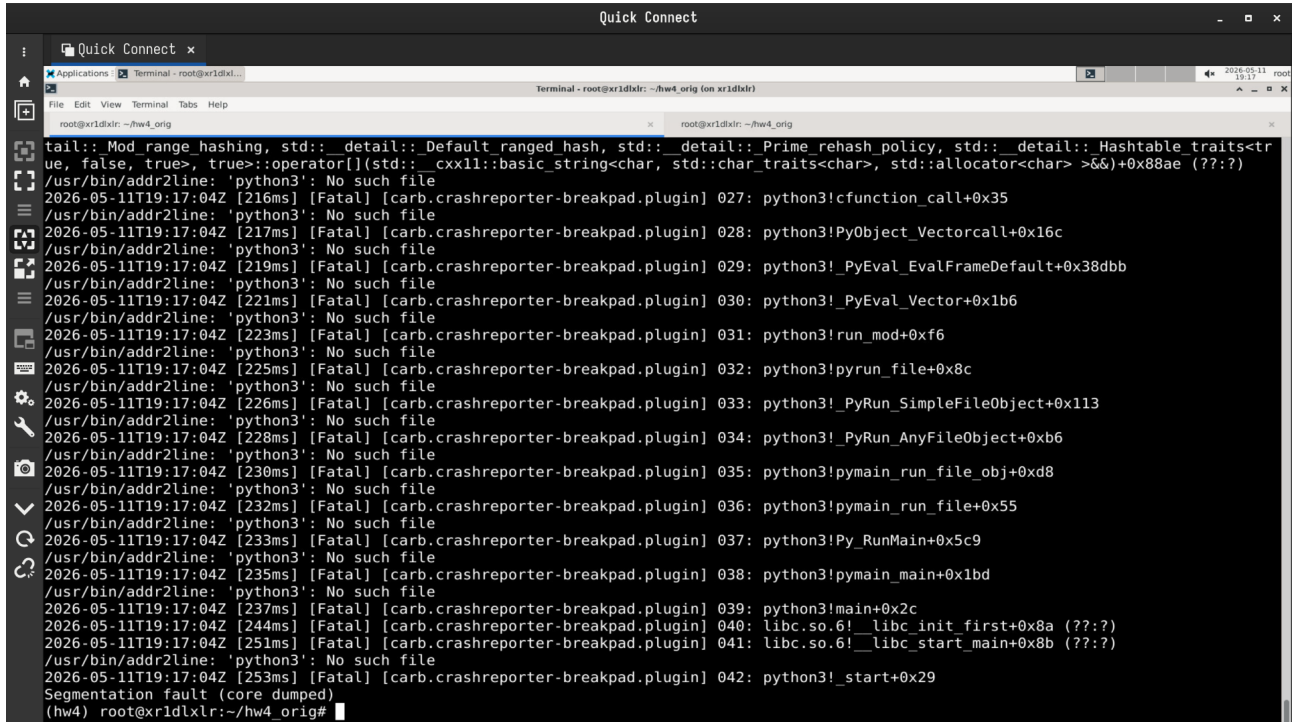
    # use the pseudo-inverse method with jacobian to calc
    # the small rotation this step
    # delta_q = step_size * (J @ error).T @ np.linalg.inv(J @ J.T)
    delta_q = step_size * (pinv(J) @ error)
    tmp_q = tmp_q + delta_q
    tmp_q = np.clip(tmp_q, joint_limits[:, 0], joint_limits[:, 1])

return list(tmp_q) # 6 DoF

```

Problems I've Encountered

The main problem of this homework are almost all about setting up the environment. First, I tried the uv environment on Glows.ai, but it just won't work and multiple segmentation error happens.



```
tail:: Mod_range hashing, std:: detail:: Default_ranged_hash, std:: detail:: Prime_rehash_policy, std:: detail:: Hashtable_traits<tr
ue, false, true>, true>::operator[](std:: cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> >&&)+0x88ae (???)
/usr/bin/addr2line: 'python3': No such file
2026-05-11T19:17:04Z [216ms] [Fatal] [carb.crashreporter-breakpad.plugin] 027: python3!cfunction_call+0x35
/usr/bin/addr2line: 'python3': No such file
2026-05-11T19:17:04Z [217ms] [Fatal] [carb.crashreporter-breakpad.plugin] 028: python3!PyObject_Vectorcall+0x16c
/usr/bin/addr2line: 'python3': No such file
2026-05-11T19:17:04Z [219ms] [Fatal] [carb.crashreporter-breakpad.plugin] 029: python3!_PyEval_EvalFrameDefault+0x38dbb
/usr/bin/addr2line: 'python3': No such file
2026-05-11T19:17:04Z [221ms] [Fatal] [carb.crashreporter-breakpad.plugin] 030: python3!_PyEval_Vector+0x1b6
/usr/bin/addr2line: 'python3': No such file
2026-05-11T19:17:04Z [223ms] [Fatal] [carb.crashreporter-breakpad.plugin] 031: python3!run_mod+0xf6
/usr/bin/addr2line: 'python3': No such file
2026-05-11T19:17:04Z [225ms] [Fatal] [carb.crashreporter-breakpad.plugin] 032: python3!pyrun_file+0x8c
/usr/bin/addr2line: 'python3': No such file
2026-05-11T19:17:04Z [226ms] [Fatal] [carb.crashreporter-breakpad.plugin] 033: python3!PyRun_SimpleFileObject+0x113
/usr/bin/addr2line: 'python3': No such file
2026-05-11T19:17:04Z [228ms] [Fatal] [carb.crashreporter-breakpad.plugin] 034: python3!PyRun_AnyFileObject+0xb6
/usr/bin/addr2line: 'python3': No such file
2026-05-11T19:17:04Z [230ms] [Fatal] [carb.crashreporter-breakpad.plugin] 035: python3!pymain_run_file_obj+0xd8
/usr/bin/addr2line: 'python3': No such file
2026-05-11T19:17:04Z [232ms] [Fatal] [carb.crashreporter-breakpad.plugin] 036: python3!pymain_run_file+0x55
/usr/bin/addr2line: 'python3': No such file
2026-05-11T19:17:04Z [233ms] [Fatal] [carb.crashreporter-breakpad.plugin] 037: python3!Py_RunMain+0x5c9
/usr/bin/addr2line: 'python3': No such file
2026-05-11T19:17:04Z [235ms] [Fatal] [carb.crashreporter-breakpad.plugin] 038: python3!pymain_main+0x1bd
/usr/bin/addr2line: 'python3': No such file
2026-05-11T19:17:04Z [237ms] [Fatal] [carb.crashreporter-breakpad.plugin] 039: python3!main+0x2c
2026-05-11T19:17:04Z [244ms] [Fatal] [carb.crashreporter-breakpad.plugin] 040: libc.so.6!__libc_init_first+0x8a (???)
2026-05-11T19:17:04Z [251ms] [Fatal] [carb.crashreporter-breakpad.plugin] 041: libc.so.6!__libc_start_main+0x8b (???)
/usr/bin/addr2line: 'python3': No such file
2026-05-11T19:17:04Z [253ms] [Fatal] [carb.crashreporter-breakpad.plugin] 042: python3!_start+0x29
Segmentation fault (core dumped)
(hw4) root@xr1d1xlr:~/hw4_orig#
```

Then, I've tried the docker method on the Glows.ai instance. However, it is time consuming and is a waste of the precious 1 point credit. It takes like half an hour without success, so I stopped it and turned to find other solutions.

Glad that, I have good friends, who know how to debug. By setting this list of environmental variables, I can successfully fix the errors and proceed:

- `export VK_ICD_FILENAMES=/etc/vulkan/icd.d/nvidia_icd.json`
→ This tells Vulkan to use Nvidia's Driver
- `export OMNI_KIT_ARGS="--/renderer/multiGpu/enabled=false --/app/renderer/activeGpu=0 --/rtx/verifyDriverVersion/enabled=false"`
→ This tells Glows.ai to use a specific single graphic card and bypass driver version checks
- `export OMNI_KIT_DISABLE_DRIVER_VERSION_CHECK=1`
- `export OMNI_DISABLE_DRIVER_CHECK=1`
→ The last two also disables version checks.

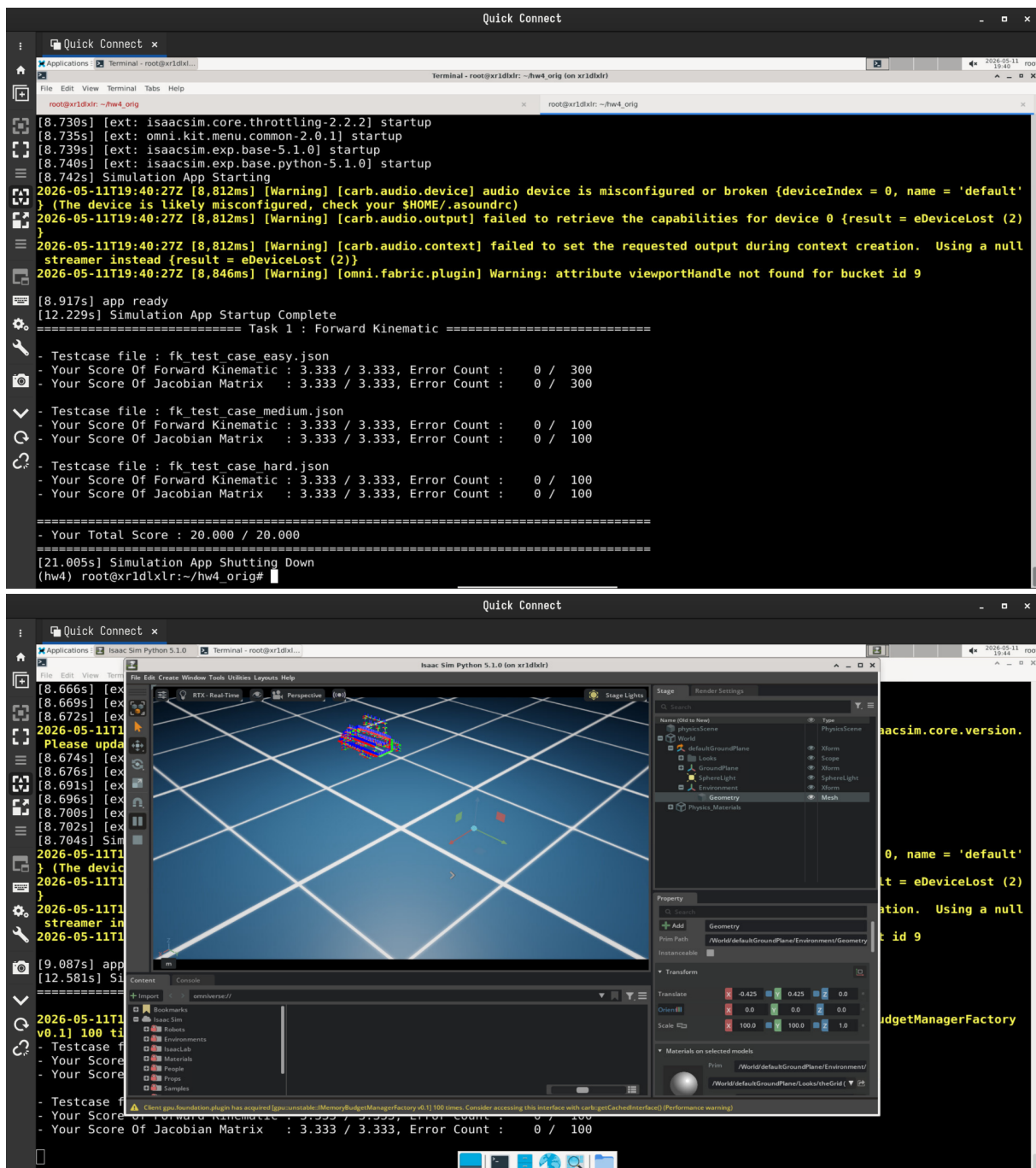
With these patches, I could succesfully run both fk.py and ik.py within minutes.

No Bonus

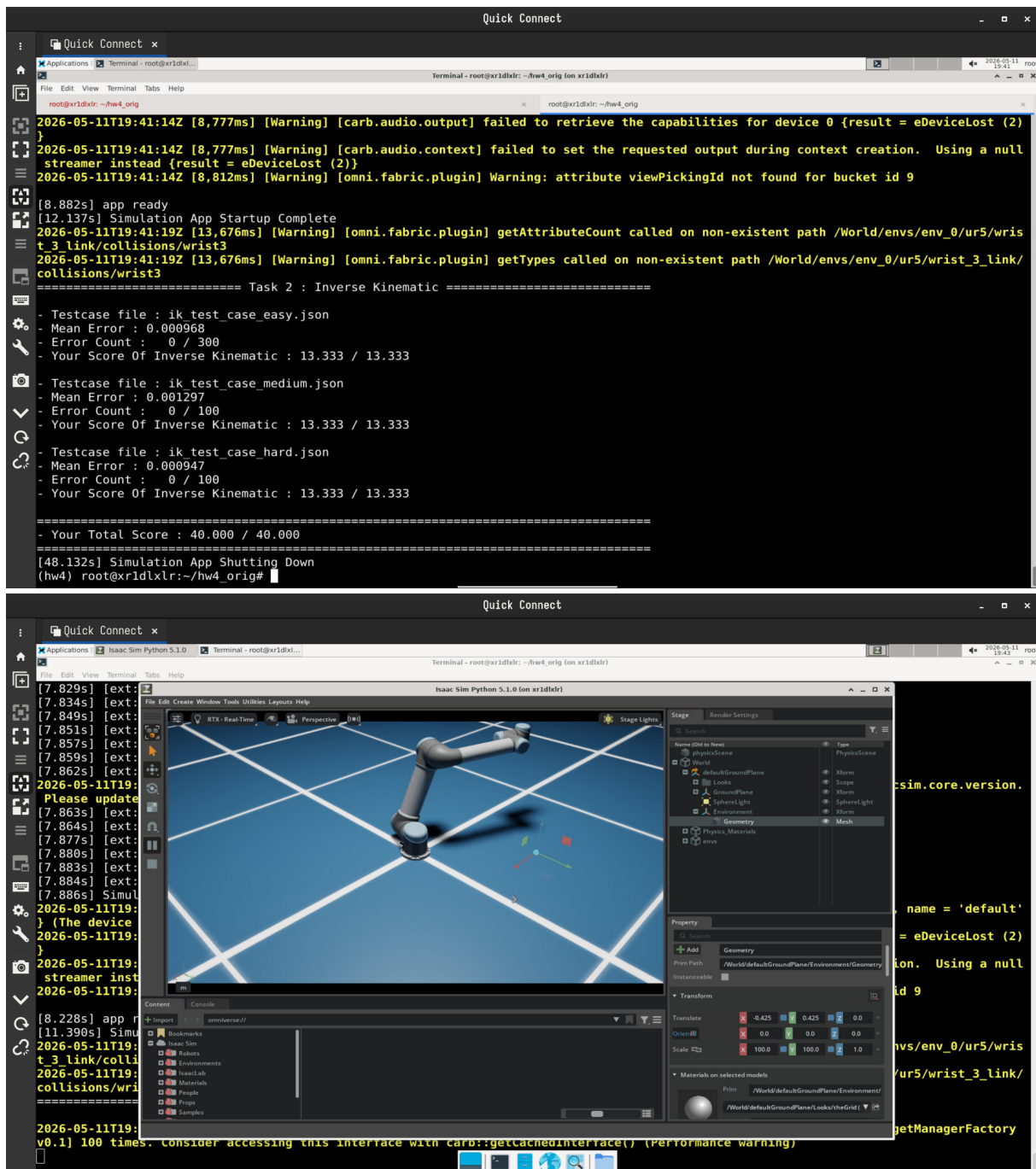
I found it actually time consuming to understand the algorithms and set up the development environment, which takes me about two whole days.

The Screenshots about Passing Task 1 & 2

These two images shows that I have successfully connected to a temporary instance on glows.ai and runs the task1's testcase under uv. The commands are (in the uv environment, .venv): "python fk.py --headless" and "python fk.py". I didn't wait till the latter stops because it is so time consuming to render GUI.



Then, these two are for task2. The commands are (also in the uv environment .venv):
 "python ik.py --headless" and "python ik.py". I also didn't wait till the GUI stops.



References

I mainly referenced these materials:

- Professor Yi-Ting Chen's recorded [Lecture 8](#)
- [The slide that's suggested to read](#) while implementing IK. The link in HW4_spec_2026 is outdated (the original file is removed), and I can only find it from Internet Archive's Wayback Machine.

- [A 3D animation of D-H](#) that's mentioned both in Professor Yi-Ting's lecture and the slides suggested to read.
- [The ur5 robot arm's URDF and STL files](#) stored in the old commits of this course's official Github repository, ai-capstone26. The file ur5.urdf is mentioned in the code template of fk.py, but it is removed in the newest version.\
- [Franka's URDF and STL files](#).
- An open-source URDF visualizer on Github. I used it to open the URDF and STL files mentioned above. [Repo Demo](#)
- [A D-H Parameter table online](#).